From PDF to Structured Data: Deploying an LLM Pipeline to Make US Early Learning Standards Machine-Readable

Anonymous submission

Abstract

US states publish early learning standards for preschool-age children mostly as PDFs, each in its own design, so this body of public policy does not exist as structured data. We report on deploying an LLM pipeline that converts these documents into one queryable schema, in which every extracted standard becomes a record that downstream applications and human reviewers depend on. Deployment imposed requirements that extraction benchmarks do not: new documents without new code, keys that are reproducible across runs of a sampling model, nothing structurally impossible persisted, bounded cost, and failures that are visible. The detector meets the first by classifying each element by its nesting position against a depth map inferred per document in a cheap first pass. On annotated subsets of six states, detector recall is 1.000 at every level in the recorded run (at least 0.978 in repeats) and an audit of 297 detections finds one hallucination; ablating the depth map degrades exactly the positional levels, and a rule-based extractor matches the method only at the top level. Inference for one state's complete standards (41–52 pages) costs about $4. Most of the remaining requirements were met by moving every rule with a single correct output out of the model and into code, and by validating at the persistence boundary. Four of the six deployment failures we report surfaced only on whole documents in production, including a run that reported success while losing most of its document.

1 Introduction

Early learning standards are the developmental expectations that shape curriculum, assessment, and teacher preparation for children before kindergarten entry. More than fifty US jurisdictions issue them for preschool-age children (National Center on Early Childhood Quality Assurance n.d.) and each designs its own document. Unlike K–12 academic standards, which have machine-readable exchange formats (Sutton and Golder 2008; 1EdTech Consortium 2017), early learning standards exist almost entirely as PDFs: multi-column tables, age-banded grids and prose whose typography is not commonly shared. Curriculum-planning tools, alignment studies and cross-state comparisons all need them as structured data, and today that data is produced by hand, if at all.

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

Turning these documents into data is a deployment problem before it is an extraction problem. The pipeline's output is not a benchmark answer but a database row whose identifier, `{country}-{state}-{year}-{indicator_code}`, is a primary key read by a state standards explorer. An indicator means something only through its ancestry (which domain, strand and sub-strand it sits under, and at what age band), so a misplaced level or a key that changes between runs is a data-integrity failure, not a scoring error. And the surface cues that reveal ancestry are exactly what varies: Arizona writes `Strand 1: Language and Literacy`; Kentucky writes `Benchmark 1.1`; Nevada prints a code namespace that skips a level; Colorado has no sub-strand tier at all. A rule tuned to one document's vocabulary fails on the next, and no annotated corpus exists to learn the mapping from.

We describe a system that recovers this structure with in-context LLM calls and runs as a serverless pipeline on AWS. Its detector classifies elements by nesting position, not label: a first, cheap pass infers a per-document depth map, and every element is then classified against that map. `Benchmark 1.1` need not be recognized as a "sub-strand", only as sitting one level below `Language and Early Literacy Standard 1`. A parser resolves the detected elements into one canonical schema with deterministic identifiers, a validator guards the database, and a human can review every stored record in an editorial interface. The system runs end to end in production infrastructure and is in pre-launch use as the data layer behind both applications.

Our central observation is that the system became deployable less by making the model more accurate than by deciding what the model is allowed to decide. It makes the judgments no rule can make across documents; everything with a single correct output (an abbreviation, a uniqueness tie-break, a structural invariant, whether a run is healthy) is taken away from it.

Contributions. (1) Deployment requirements for LLM extraction whose output is stored as records that applications depend on, and a system that meets them with no per-state code path: two-pass detection by per-document depth map, and a parser that resolves six independently designed documents into one schema with reproducible keys (§3–4). (2) An evaluation on six states covering quality, an ablation, a rule-

based baseline, run-to-run stability, and whole-document scale and cost (§5). (3) A record of six deployment failures, where each surfaced and what now guards it, distilled into practices for deploying LLMs whose outputs are stored as data (§6).

2 Related Work

Document hierarchy. Layout analysis and OCR-free transcription recover text and page objects (Lopez 2009; Zhong, Tang, and Jimeno Yepes 2019; Xu et al. 2020; Huang et al. 2022; Kim et al. 2022; Blecher et al. 2023), and hierarchical document parsing reconstructs logical trees by learning from annotated renderings of a genre with consistent typography (Rausch et al. 2021; Ma et al. 2023; Wang et al. 2024; Xing et al. 2024; Wang, Hu, and Huo 2025). Our setting has neither. Training-free neighbors segment law books from table-of-contents cues (Wehnert, Changaramkulath, and Habernal 2025) or prompt for a pseudo table of contents for retrieval (Jeong et al. 2025); our depth map is the table-of-contents subtask of Wang et al. (2024), inferred zero-shot and used as the authority for every classification.

LLM extraction without labels. Generative extraction is competitive where labeled data is scarce (Agrawal et al. 2022; Wadhwa, Amir, and Wallace 2023; Xu et al. 2024), though fine-tuning wins where labels exist (Liu et al. 2022). GoLLIE (Sainz et al. 2024) trains models to follow annotation guidelines; our prompts state document-agnostic structural guidelines to a frozen model. We validate JSON and repair it deterministically rather than constraining decoding (Willard and Louf 2023), chunk by domain given long-context degradation (Liu et al. 2024), and audit for hallucination (Ji et al. 2023). Human effort goes to blanket verification rather than to annotation or confidence-guided review (Monarch 2021; Settles 2009; Li et al. 2023).

Deploying learned systems. Production ML accumulates failure modes that sit outside the model (Sculley et al. 2015; Paleyes, Urma, and Lawrence 2022), and readiness rubrics ask for validation of data at system boundaries and for parity between the code that is tested and the code that serves (Breck et al. 2017, 2019). LLMs add output that varies across identical requests (Ouyang et al. 2023). Our validator applies boundary validation to model output, as Aggarwal et al. (2025) do for fiscal tables, and our lessons report concrete instances of these risks in an LLM pipeline whose outputs are stored as data. Unlike those case studies, the pipeline has no training step: every component is a prompt, a frozen model or deterministic code, so a model update is a prompt edit, and we key every recorded result to a hash of the source that holds the prompts.

Machine-readable standards. ASN (Sutton and Golder 2008) and CASE (1EdTech Consortium 2017) model and exchange K–12 standards, and alignment work assumes a normalized set already exists (Porter 2002; Polikoff, Porter, and Smithson 2011; Devaul, Diekema, and Ostwald 2011; Xu et al. 2026). For early childhood, it does not: the one federal framework binds only Head Start (Office of Head Start 2015). This paper automates that missing input.

3 Deployment Setting and Requirements

Canonical schema. The target schema has four levels, *domain*, *strand*, *sub-strand* and *indicator*, of which the middle two are optional, because a four-level schema is not a four-level document: Colorado publishes no sub-strand tier, and forcing one would require inventing structure. Each level has a code, a name and an optional description; the indicator also carries an age band, a source page and the verbatim source text. The unit of storage is a leaf with its full lineage, keyed by the identifier above, whose indicator code is fully qualified and carries any side-by-side disambiguator itself: an age column appends its month range (Texas `I.A.2.36-48` vs. `I.A.2.48-60`, and likewise California's age columns), and a proficiency column its label (California `ELD.1.0.VOCA.1.1.DISC`). Every level also carries human-verification state (Appendix B).

Corpus and tiers. We use documents from six states: four for development (AZ, CA, CO, TX) and two for validation (NV, KY). No code path names any state, but validation-state failures prompted several general fixes during development, so the set is *not* a clean held-out test (§7). Each document exists at up to three tiers. The *full* tier is the published PDF. The *trimmed* tier removes front matter, essays, and appendices but **retains every standard**, so it reduces cost but not coverage. The *subset* tier is one to two domains per state, 70 pages and 262 standards in total, annotated for grading (§5); **all headline quality numbers are at this tier**. Both cuts were made by hand, which favors the system; Appendix E lists every source document and the retained pages.

Requirements. Five requirements came from deployment rather than from the extraction task, and organize the rest of the paper: **R1** a new state's document is processed without new code; **R2** a standard's key is reproducible from the document, not from the order or the run in which the pipeline processed it; **R3** no structurally impossible record is persisted, and every stored record is cheap for a human to check against its source; **R4** a whole document runs within serverless time limits and model quotas at a cost of a few dollars; **R5** a run that loses data says so. R1–R4 are met by the design of §4; R5 is only partly met, and §6 says why.

4 System

The pipeline has six stages (Figure 1). *Ingestion* registers a PDF; *extraction* recovers text blocks with page coordinates from Textract OCR repaired against the PDF's own text layer; *detection* emits a flat list of typed elements; *parsing* resolves them into the schema; *validation* checks structural invariants; *persistence* writes surviving records to PostgreSQL. Stages are Lambda functions that exchange JSON through S3, so any stage can be replayed in isolation from the previous stage's output. Detection and parsing are batched through a prepare → Step Functions `Map` → merge pattern (at most three concurrent workers) so that a long document does not exhaust a function timeout (R4).

Two stages, two questions. Detection answers *what is on the page*: a local judgment. Parsing answers *what it is a child of*: a global one, since a parent may be printed pages earlier.

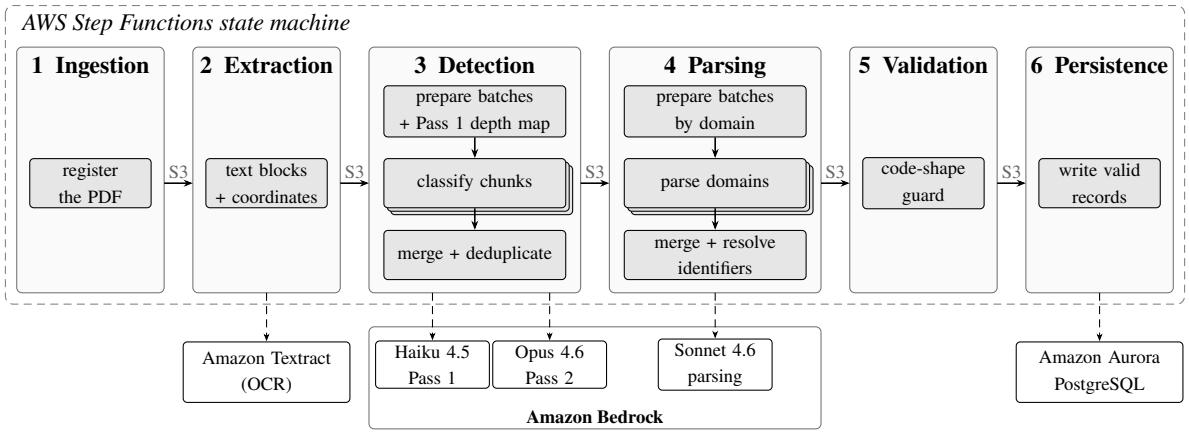


Figure 1: Deployed architecture, read left to right. Each gray box is one of the ten AWS Lambda handlers; stacked boxes fan out through a Step Functions `Map` state with at most three concurrent workers. Every hand-off between stages is a JSON write and read through Amazon S3. Dashed arrows are calls to external services; only records that pass validation are persisted.

Each stage sees the context it needs (parsing reasons over a whole domain at once), is graded against its own golden, and runs on the cheapest adequate model (R4): the depth-map pass on Claude Haiku 4.5, detection on Claude Opus 4.6, parsing on Claude Sonnet 4.6, all at temperature zero.

4.1 Detection

Pass 1: depth map. One call infers how many nesting levels the document uses and how each is rendered (Appendix A.1). Every element's level is later assigned against this map, so *a level missing from Pass 1's input is missing from the whole run*. The pass sees a 6,000-token sample, and sampling matters: headings are rare and prose abundant, so every-nth sampling drops the lines that establish the top of the hierarchy (§6). We instead bucket blocks by their normalized left edge, since indentation tracks nesting and a rare x-position is a rare depth; each bucket receives a floor of twelve blocks spread across the document, rarest buckets first, and the remaining budget goes to the bulk.

Pass 2: classification. The document is chunked into ∼2,000 token windows with 500 tokens of overlap, and each chunk is classified with the depth map as context. The central instruction is positional: classify an element by where it sits in the map, not by its label word (Appendix A.2). Supporting rules are equally general (R1): in a `<Label> <id>: <Title>` heading, for *any* label word, the label and id are the code; a printed code is authoritative and may appear inline, in a caption, or as the shared prefix of the element's descendants' codes. If the document does not print a code, one is derived from the title by a fixed abbreviation procedure (Appendix A.2).

4.2 Parsing and identifiers

Parsing composes each indicator's qualified code from its ancestors, and printed namespaces are not uniform. Nevada's sub-strand code does not extend its strand's, so requiring it would corrupt all 24 Nevada standards; Kentucky's `Benchmark 1.1` restates its parent inside its own id, so naive composition counts the parent twice. Each case is handled by a rule over *shape* rather than vocabulary: first peel an ancestor's code from a descendant's until the namespace stops; then keep a level's own id when its peeled code matches the level below it; next do not re-qualify an already-qualified code; then collapse a dotted child that repeats its parent's last segment; and finally decline any repair that cannot be verified against the row's own ancestry, or would introduce whitespace (Appendix A.3).

Uniqueness (R2) is enforced after the batch merge, since collisions span batches: colliding rows are re-qualified with their own ancestors' segments, all members are rewritten so the result is order-independent, and a numeric suffix in document order is a logged last resort. Finally, the validator (R3) rejects any record whose codes contain whitespace, whose indicator code does not extend its nearest ancestor's, or whose identifier does not end in that code. Rejected records never reach the database; every stored record carries its source page and per-level verification state for review.

4.3 Design discipline: prompts for judgment, code for shape

An earlier revision reached strong scores on the development states by accumulating per-state logic: regular expressions for label words, a collision special case, and an abbreviation helper tuned to one state, and it transferred poorly to new documents. That is a direct violation of R1, so we removed that logic and adopted a rule: keep structural decisions in the prompt as general document principles; Python may only read the *shape* of an output (dot segments, whitespace, containment, layout position), never a document's words. A new per-state branch is a regression even if it raises a score.

Deterministic code survives only where a prompt rule cannot reach zero error, and it is *paired* with that rule rather than replacing it: the code executes the same rule the prompt states, never a competing one, so the prompt lowers the error rate and the code makes its floor zero (§6, practice 1). The prompts are not free of the corpus: fifteen of their worked

			Detection		Parsing		
State	**pp**	**Std.**	**Ver. prec.**	**Code acc.**	**Coverage**	**Field acc.**	**Fully corr.**
AZ	15	45	1.000	4/4	18/18	0.997	17/18
CA	13	94	1.000	25/25	21/21	1.000	21/21
CO	10	48	1.000	7/7	9/9	1.000	9/9
TX	9	25	1.000	8/8	8/8	1.000	8/8
NV	15	24	0.981	46/46	24/24	1.000	24/24
KY	8	26	1.000	44/44	26/26	1.000	26/26

Table 1: Quality per state at the *subset* tier (§3); validation states below the rule. *pp*: subset pages; *Std.*: standards parsed; *Ver. prec.*: verified precision (§5). One recorded run; its recall (1.000) and Pass-1 level count (correct) are uniform and omitted. Over repeats (§5.3), NV detector recall is 45–46/46, code accuracy 44–45/46, and 4 NV parser strand codes vary.

examples are distinctive to one state's golden, including four Kentucky strings and one Nevada string illustrating the abbreviation procedure.

5 Evaluation

Protocol. Two suites grade the stages separately: the detector against flat-element goldens, and the parser against nested normalized-standard goldens. Development-state detector goldens are spot checks (5–25 elements), so an unmatched detection is usually an annotation gap rather than an error; raw precision there measures annotation coverage. We therefore report *verified precision* from a manual audit of every in-scope detection the golden does not match (162 of the 297 in-scope detections), counting as errors only elements absent from the source. Kentucky's golden is exhaustive, and its precision is a true hallucination rate. The suites exercise the direct, unbatched path; the batched production path is evaluated separately at scale (§5.4). Every tabulated number is regenerated by script from a recorded run artifact.

5.1 Quality on six states

Table 1 gives the results. In the recorded run, detector recall is 1.000 at every level in all six states, including both validation states (R1). Over 135 pooled golden elements that gives a 95% lower bound of 0.973 (Clopper–Pearson), and Nevada's recall falls to 45/46 in some repeats (§5.3). Pass 1 infers the annotated level count in all six, including Colorado's three. The audit finds one hallucinated element (verified precision 0.997 pooled): a Nevada indicator emitted a second time from a truncated transcription of a multi-column table, with an invented clause appended. The parser covers every annotated standard, and its uniqueness resolver, which logs when it acts, never had to separate two standards in the 29 subset-tier runs whose logs were retained.

5.2 Ablation and baselines

Depth map. Disabling Pass 1 makes the detector fall back to the no-map path it already uses when inference fails. In Table 2, domain and indicator recall are untouched, while strand falls to 0.920 and sub-strand to 0.833. The levels that survive have distinctive surface forms (a document's top division; its numbered leaves), and the levels that degrade are exactly those identifiable only by what they sit between. On Kentucky, the map-on arm reproduces the golden's 3/5/10/26 elements per level exactly, while the map-off arm emits 4/10/6/26: `Benchmark N.N` items are promoted to strand because of their label. The direction is fixed across three runs per arm per state, but the magnitude is a draw (Kentucky off-arm recall 0.89–0.98; Colorado 0.71–0.86); Texas and Nevada keep recall and lose code accuracy instead (Texas 8/8 → 5/8).

Level	n	**Full**	**No map**	**Rule-based**
domain	13	1.000	1.000	1.000
strand	25	1.000	0.920	0.240
sub-strand	24	1.000	0.833	0.667
indicator	73	1.000	1.000	0.137

Table 2: Detector recall by hierarchy level, pooled over six states: the full method, the method with the Pass-1 depth map ablated, and a rule-based extractor graded by the same suite. Subset tier; ablation off-arm recall varies by run (§5.2).

Rule-based extractor. A deterministic extractor built on numbering patterns, capitalization, indentation, and line shape, developed against the four development states and graded by the same suite, ties the LLM at domain (13/13) and collapses below it (Table 2). Its misses are omissions, not mislabelings: on Kentucky it never emits 31 of 44 golden elements and misplaces none. Even on the development states, its recall is 0.57–0.80.

One whole-document prompt. One call per document, with the same prompt, model and grader but no depth map, chunking or repairs, matches the method on Arizona and Nevada. It loses code accuracy on Kentucky (44/44 → 28/44) and Texas (8/8 → 5/8), each miss a different primary key for the same standard, and recall on Colorado (0.429). California fails entirely: its call stopped at exactly the 16,000-token output cap, truncating the JSON so nothing parsed. Arizona succeeded within 11% of the same cap. Chunking bounds such a loss to one chunk; at full-document scale it is not optional. This arm is one draw per state.

5.3 Stability

We re-ran the detector five times and the parser six times per state on frozen inputs with caching disabled, split across

	KY	CO
Pages	52	41
Detection chunks / batches	18 / 4	17 / 4
Parse batches	9	9
Elements raw → merged	363→329	390→300
LLM calls	45	45
Input / output tokens (k)	379 / 150	390 / 131
Wall clock	13m19s	9m52s
Inference cost (USD)	**$4.41**	**$4.02**
depth map (Haiku 4.5)	0.3%	0.5%
detection (Opus 4.6)	59%	57%
parsing (Sonnet 4.6)	40%	42%

Table 3: One batched production run per document at the *trimmed* tier: every standard in the document, front matter and essays removed (§3). Cost is model inference only, from recorded tokens at Bedrock on-demand rates confirmed in August–September 2026. Textract OCR ($0.015 per page, $0.78 and $0.61 here) and fixed infrastructure are excluded.

two days. Instability is concentrated: in the pooled runs, 5 of 304 detected elements and 9 of 201 parsed indicators differ in a graded field, all in four places. Arizona's detector intermittently omits three sub-strands outside the golden; Nevada's domain code alternates between `S` and `Science`; Nevada's parser places 4 indicators under a sibling strand (e.g., `SS.1` for `SS.2`), each in one run; and California's parser intermittently leaves a structural label inside an indicator code (`ELD.1.0.VOCA.Foundation 1.1.BROA`), on 12 rows across 4 indicators. All 12 malformed codes contain whitespace, and the validator rejected every one before persistence (R3); the Nevada strand codes are well formed, so no shape check can catch them. Splitting across days mattered: Nevada's detector was identical in three runs one evening and dropped an unannotated indicator in both runs the next day; in five further runs it missed one golden indicator twice (recall 45/46). Nevada's recorded detector code accuracy of 46/46 did not reproduce: five fresh runs give 44–45. These rates are lower bounds.

5.4 Scale and cost

The subset tier never exercises batching (every document fits in one batch). Table 3 reports one production run each on Kentucky's and Colorado's complete standards (trimmed tier): 17–18 detection chunks in four batches, merges that remove 34 and 90 duplicate elements, about $4 of model inference (Textract OCR adds $0.6–0.8), and 10–13 minutes per document, with no throttling or rejected records (R4). Detection dominates cost (57–59%), while the depth-map pass costs 0.3–0.5% of a run, one to two cents. The limit we hit in practice was neither Lambda time nor concurrency but a per-model *daily* token quota on the detection model, exhausted by evaluation sweeps‡; running work in sequence does not avoid it, so capacity has to be budgeted across days.

To grade quality at this scale, we annotated Kentucky's complete trimmed tier (277 elements, 207 standards): the annotation started from one run's output, every element was checked against the published PDF, and the PDF was searched for elements no run emitted. On a document with $6.5\times$ the subset's page count, three further runs each find all **277 of 277** elements and emit no ungrounded element. Scale does introduce one defect the subset never shows. Kentucky prints a `THREE AND FOUR YEAR OLDS` page banner, which the detector attaches to 164 elements as their own age band in each of three runs (159 of them the same elements). Deduplication keys on age band, so ∼15% of detector output survives as banded/unbanded duplicates, which collide in parsing and receive numeric-suffix identifiers: 10–11 fabricated primary keys per run, naming standards that do not exist. Parser field accuracy holds at 0.995 ± 0.002, but coverage ranges 0.81–0.93. Colorado, which has no banner, shows zero duplicates and zero fabricated keys over 240 standards.

Failure	Subset?	Now
1. Model samples a deterministic abbreviation; 58% of keys move across runs‡	yes	executed in code
2. Composed code keeps a label or drops its chain	yes	rejected at validation
3. Run `SUCCEEDED` with 12 of 18 chunks lost‡	no	per-element isolation; detection is manual
4. Uniqueness step absent from the batched path‡	no	wired in; integration test
5. Pass 1 loses a level on a whole document‡	no	layout-stratified sample
6. Page banner read as age band; fabricated keys	no	open, documented

Table 4: The six deployment failures of §6, numbered as the practices that address them. *Subset?*: whether the failure appeared on the annotated subset tier that the quality suites grade. Four of six did not.

6 Deployment Failures and Practices

Table 4 lists the failures that shaped the deployed system; each practice below addresses the failure with its number. Figures marked ‡ come from production logs and engineering notes kept during development, not from the recorded evaluation artifacts behind §5.

1. Move anything with one correct output out of the model (R2). The title-abbreviation procedure (§4.1) is a deterministic string algorithm, and a temperature-zero model samples it: across three detector runs on one frozen Kentucky extraction, 11 of 44 elements received a different code at least once‡, while level, page and age band never varied; 58% of Kentucky `standard_id`s, which are database keys, moved across three parser runs‡. We now execute the procedure in Python, but only where the document printed no code, as determined by two shape guards: the code is uppercase letters only, and it does not appear in the element's own source text. It reproduces all 40 hand-annotated derived codes. The model makes judgments; anything with a single correct output moves into code.

2. Enforce correctness where data is persisted, not by agreement (R3). Composed codes are right most of the time, which is not good enough for a key, so the validator rejects structurally impossible records. It had zero false positives on all 262 subset standards and blocked every historical defect row‡, and in our stability runs it caught every malformed California code (§5.3). Its rules are deliberately scoped: requiring every level, not just the leaf, to extend its parent would reject all of Nevada. Rejection can also signal an error elsewhere: when Pass 1 lost a level on Kentucky's complete document (failure 5), the validator rejected 102 of 202 standards‡ because codes had nothing to be qualified against. The validator was right; the fault lay two stages upstream.

3. Treat "succeeded" as a claim to verify (R5). A temperature-zero model intermittently returns `"code": null` for a required field, and retries cannot help because they resend the identical prompt. On one Kentucky production run, one such element per chunk raised a schema error inside the per-element loop, discarding every valid element in its chunk, and the worker's three retries failed identically. The execution reported `SUCCEEDED`, with no Step Functions retries and no throttling, while 12 of 18 detection chunks were lost, and records were persisted from only 40% of the document‡. The fixes were small (derive an absent code; isolate per-element failures), but stage status still does not propagate into execution status. We now check element and page coverage by hand after every run; automating that check is open work, and is why R5 is only partly met.

4. Evaluate the path you deploy. Evaluation runs detection and parsing directly; production batches them. The uniqueness resolver was called on the direct path but never in the batched merge, whose comment claimed otherwise, so a Kentucky run persisted two distinct indicators under one key‡. Conversely, a fix for descriptions split across chunk overlaps does nothing in evaluation (one chunk, one view) and fully restores a 3,500-character Nevada description in production‡. The batched merge now runs the direct path's final steps in the same order, guarded by an integration test that places colliding rows in *separate* batches.

5. Evaluate on whole documents, not only subsets. The every-nth Pass-1 sampler worked on every subset document, because each fell under the token budget and was never sampled. On Kentucky's complete document it kept 2 of 9 section headings, Pass 1 reported three levels for a four-level document, and zero sub-strands were emitted among 267 elements‡. With layout-stratified sampling, Pass 1 keeps 9 of 9 and reports four levels stably across three runs‡. The age-band defect (failure 6, §5.4) is the same lesson: whole documents must be in the evaluation, not only in production.

6. Review everything; do not gate on confidence. Detection emits a confidence score, but across 379 elements it takes five distinct values, all in $[0.90, 0.97]$; the band the prompt reserves for guesses is never used; and the lowest-confidence state, Kentucky, is perfect on recall, code accuracy and precision (Appendix D). A threshold would gate on noise, so nothing reads the score. Instead every record carries its source page and per-level verification state, and review is blanket. The open age-band defect shows why: its fabricated keys pass every check the validator makes, and review against the source page is what catches them.

7 Limitations

Deployment status and scope. The system is pre-launch: we report engineering and quality evidence, not usage, reviewer effort, or downstream outcomes. The corpus is six English-language US documents, and we make no claim about other document families or countries. No document as large as Arizona's 217-page publication has been run end to end.

Evidence. Headline quality is measured on hand-trimmed subsets (70 pages); whole-document quality is measured for one document (Kentucky). Nevada and Kentucky were checked against during development, their failures motivated several general rules, and the prompts carry five strings from their goldens, so they are validation results, not a held-out test. Development-state detector goldens are spot checks, so only Kentucky provides a precision figure that is a true hallucination rate; all goldens and the hallucination audit are by a single annotator, so no inter-annotator agreement is reported, and pooled recall rests on 135 golden elements. All comparisons are internal: the depth-map ablation, a rule-based extractor of our own, and a single-call prompt run once per state; no off-the-shelf document parser was evaluated. All stages use one model family; a model-tier ablation and other providers are untested.

Reproducibility. The pipeline is not deterministic; stability rates are lower bounds from five or six runs, and one of our earlier recorded results did not reproduce. The quality suites grade the direct path; the batched production path is graded only at scale. Figures marked ‡ in §6 come from engineering logs and are not backed by archived evaluation artifacts.

Open defects. The age-band defect of §5.4 is deliberately not repaired. Every recorded result is keyed to a hash of the detector and parser source, and the fix, a prompt rule that a band comes from a column distinguishing siblings and never from a page banner, would invalidate every measurement reported here; deduplication should also stop keying on so unstable a field. Stage status does not propagate into execution status, so a run that loses data can still report success, and coverage is checked by hand (§6, practice 3). A heading that OCR fuses or drops cannot be recovered downstream; the one hallucination arose this way.

8 Conclusion

Classifying elements by nesting position rather than by label recovers the hierarchy of six independently designed early learning standards documents with no per-state code: recall 1.000 at every level in the recorded run, one hallucination in 297 audited detections, and about $4 of model inference per document. What made it deployable was a division of labor: the model makes the judgments that differ across documents, and code owns everything with one right answer (keys, invariants, run health). Our worst failures, a lost hierarchy level,

most of a document discarded behind a `SUCCEEDED` status, and fabricated primary keys, never appeared on the evaluation subsets; a deployable LLM system has to be evaluated as it runs. The next test is a state seen only after development is frozen; the goal is every jurisdiction.

Ethical Statement

Data. The inputs are published policy documents from state agencies. No student, teacher or family data and no human subjects are involved; the only people recorded are authenticated editors, whose identity comes from their session, never from a model.

Stewardship. Standards shape curriculum and assessment, so a misextracted standard can propagate into materials that reach children. The design therefore assumes a human will check every record and makes checking cheap (source page on every record, per-level verification state), and the normalized records are pointers into the authoritative documents, not replacements for them. Using unverified output for anything with regulatory or assessment weight would be a misuse. A queryable corpus also invites cross-state comparisons the documents do not support: a state with fewer recorded indicators does not have a lower bar.

Artifacts. State government works have no uniform public-domain status, so we do not redistribute the source documents or the goldens derived from them; Appendix E identifies each document so it can be obtained from its agency. The complete prompts are provided as supplementary material, and the schema and annotated examples are in the appendices. The implementation is not released, which limits reproduction to reimplementing the method from the prompts and schema.

References

1EdTech Consortium. 2017. Competencies and Academic Standards Exchange (CASE) Specification. https://www.1edtech.org/standards/case. Version 1.0 (2017); version 1.1 released 2025. Accessed August 2026.

Aggarwal, V.; Kulkarni, J.; Mascarenhas, A.; Narang, A.; Raman, S.; Shah, A.; and Thomas, S. 2025. Information Extraction From Fiscal Documents Using LLMs. *arXiv preprint arXiv:2511.10659*. Presented at the AI for Financial Inclusion, Risk Modeling and Resilience in Emerging Markets workshop at ACM ICAIF 2025.

Agrawal, M.; Hegselmann, S.; Lang, H.; Kim, Y.; and Sontag, D. 2022. Large Language Models are Few-Shot Clinical Information Extractors. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, 1998–2022. Abu Dhabi, United Arab Emirates.

Blecher, L.; Cucurull, G.; Scialom, T.; and Stojnic, R. 2023. Nougat: Neural Optical Understanding for Academic Documents. *arXiv preprint arXiv:2308.13418*.

Breck, E.; Cai, S.; Nielsen, E.; Salib, M.; and Sculley, D. 2017. The ML Test Score: A Rubric for ML Production Readiness and Technical Debt Reduction. In *Proceedings of the IEEE International Conference on Big Data*, 1123–1132.

Breck, E.; Polyzotis, N.; Roy, S.; Whang, S. E.; and Zinkevich, M. 2019. Data Validation for Machine Learning. In *Proceedings of Machine Learning and Systems (SysML)*.

Devaul, H.; Diekema, A. R.; and Ostwald, J. 2011. Computer-Assisted Assignment of Educational Standards Using Natural Language Processing. *Journal of the American Society for Information Science and Technology*, 62(2): 395–405.

Huang, Y.; Lv, T.; Cui, L.; Lu, Y.; and Wei, F. 2022. LayoutLMv3: Pre-training for Document AI with Unified Text and Image Masking. In *Proceedings of the 30th ACM International Conference on Multimedia*, 4083–4091.

Jeong, H. S.; Jo, S.; Yoon, B. H.; Heo, Y.; Jeong, H.; and Kim, T. 2025. Zero-Shot Document Understanding using Pseudo Table of Contents-Guided Retrieval-Augmented Generation. *arXiv preprint arXiv:2507.23217*.

Ji, Z.; Lee, N.; Frieske, R.; Yu, T.; Su, D.; Xu, Y.; Ishii, E.; Bang, Y. J.; Madotto, A.; and Fung, P. 2023. Survey of Hallucination in Natural Language Generation. *ACM Computing Surveys*, 55(12): 248:1–248:38.

Kim, G.; Hong, T.; Yim, M.; Nam, J.; Park, J.; Yim, J.; Hwang, W.; Yun, S.; Han, D.; and Park, S. 2022. OCR-Free Document Understanding Transformer. In *Computer Vision – ECCV 2022*, 498–517. Springer.

Li, M.; Shi, T.; Ziems, C.; Kan, M.-Y.; Chen, N. F.; Liu, Z.; and Yang, D. 2023. CoAnnotating: Uncertainty-Guided Work Allocation between Human and Large Language Models for Data Annotation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 1487–1505. Singapore.

Liu, H.; Tam, D.; Muqeeth, M.; Mohta, J.; Huang, T.; Bansal, M.; and Raffel, C. 2022. Few-Shot Parameter-Efficient Fine-Tuning is Better and Cheaper than In-Context Learning. In *Advances in Neural Information Processing Systems*, volume 35, 1950–1965.

Liu, N. F.; Lin, K.; Hewitt, J.; Paranjape, A.; Bevilacqua, M.; Petroni, F.; and Liang, P. 2024. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics*, 12: 157–173.

Lopez, P. 2009. GROBID: Combining Automatic Bibliographic Data Recognition and Term Extraction for Scholarship Publications. In *Research and Advanced Technology for Digital Libraries (ECDL 2009)*, 473–474. Springer.

Ma, J.; Du, J.; Hu, P.; Zhang, Z.; Zhang, J.; Zhu, H.; and Liu, C. 2023. HRDoc: Dataset and Baseline Method Toward Hierarchical Reconstruction of Document Structures. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, 1870–1877.

Monarch, R. 2021. *Human-in-the-Loop Machine Learning: Active Learning and Annotation for Human-Centered AI*. Shelter Island, NY: Manning.

National Center on Early Childhood Quality Assurance. n.d. Early Learning Standards and Guidelines. https://childcareta.acf.hhs.gov/sites/default/files/state_elgs_web_final_2.pdf. Office of Child Care, Administration for Children and Families, U.S. Department of Health and Human Services. Accessed August 2026.

Office of Head Start. 2015. Head Start Early Learning Outcomes Framework: Ages Birth to Five. https://headstart.gov/school-readiness/article/head-start-early-learning-outcomes-framework. U.S. Department of Health and Human Services. Accessed August 2026.

Ouyang, S.; Zhang, J. M.; Harman, M.; and Wang, M. 2023. LLM is Like a Box of Chocolates: The Nondeterminism of ChatGPT in Code Generation. *arXiv preprint arXiv:2308.02828*.

Paleyes, A.; Urma, R.-G.; and Lawrence, N. D. 2022. Challenges in Deploying Machine Learning: A Survey of Case Studies. *ACM Computing Surveys*, 55(6): 114:1–114:29.

Polikoff, M. S.; Porter, A. C.; and Smithson, J. 2011. How Well Aligned Are State Assessments of Student Achievement with State Content Standards? *American Educational Research Journal*, 48(4): 965–995.

Porter, A. C. 2002. Measuring the Content of Instruction: Uses in Research and Practice. *Educational Researcher*, 31(7): 3–14.

Rausch, J.; Martinez, O.; Bissig, F.; Zhang, C.; and Feuerriegel, S. 2021. DocParser: Hierarchical Document Structure Parsing from Renderings. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, 4328–4338.

Sainz, O.; García-Ferrero, I.; Agerri, R.; Lopez de Lacalle, O.; Rigau, G.; and Agirre, E. 2024. GoLLIE: Annotation Guidelines Improve Zero-Shot Information-Extraction. In *The Twelfth International Conference on Learning Representations*.

Sculley, D.; Holt, G.; Golovin, D.; Davydov, E.; Phillips, T.; Ebner, D.; Chaudhary, V.; Young, M.; Crespo, J.-F.; and Dennison, D. 2015. Hidden Technical Debt in Machine Learning Systems. In *Advances in Neural Information Processing Systems*, volume 28, 2503–2511.

Settles, B. 2009. Active Learning Literature Survey. Computer Sciences Technical Report 1648, University of Wisconsin–Madison.

Sutton, S. A.; and Golder, D. 2008. Achievement Standards Network (ASN): An Application Profile for Mapping K-12 Educational Resources to Achievement Standards. In *Proceedings of the International Conference on Dublin Core and Metadata Applications (DC-2008)*, 69–79. Berlin, Germany: Dublin Core Metadata Initiative.

Wadhwa, S.; Amir, S.; and Wallace, B. C. 2023. Revisiting Relation Extraction in the Era of Large Language Models. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 15566–15589. Toronto, Canada.

Wang, J.; Hu, K.; and Huo, Q. 2025. UniHDSA: A Unified Relation Prediction Approach for Hierarchical Document Structure Analysis. *Pattern Recognition*, 165: 111617.

Wang, J.; Hu, K.; Zhong, Z.; Sun, L.; and Huo, Q. 2024. Detect-Order-Construct: A Tree Construction Based Approach for Hierarchical Document Structure Analysis. *Pattern Recognition*, 156: 110836.

Wehnert, S.; Changaramkulath, H.; and Habernal, I. 2025. HiPS: Hierarchical PDF Segmentation of Doctrinal Legal Books. *arXiv preprint arXiv:2509.00909*. Accepted as a demo paper at ICAIL 2026.

Willard, B. T.; and Louf, R. 2023. Efficient Guided Generation for Large Language Models. *arXiv preprint arXiv:2307.09702*.

Xing, H.; Cheng, C.; Gao, F.; Shao, Z.; Yu, Z.; Bu, J.; Zheng, Q.; and Yao, C. 2024. DocHieNet: A Large and Diverse Dataset for Document Hierarchy Parsing. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 1129–1142. Miami, Florida, USA.

Xu, D.; Chen, W.; Peng, W.; Zhang, C.; Xu, T.; Zhao, X.; Wu, X.; Zheng, Y.; Wang, Y.; and Chen, E. 2024. Large Language Models for Generative Information Extraction: A Survey. *Frontiers of Computer Science*, 18(6): 186357.

Xu, Q.; Jiao, H.; Zhou, T.; Li, M.; Zhang, N.; Peters, S.; and Fu, Y. 2026. Automated Alignment of Math Items to Content Standards in Large-Scale Assessments Using Language Models. *Educational Measurement: Issues and Practice*, 45(3): e70038.

Xu, Y.; Li, M.; Cui, L.; Huang, S.; Wei, F.; and Zhou, M. 2020. LayoutLM: Pre-training of Text and Layout for Document Image Understanding. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 1192–1200.

Zhong, X.; Tang, J.; and Jimeno Yepes, A. 2019. PubLayNet: Largest Dataset Ever for Document Layout Analysis. In *2019 International Conference on Document Analysis and Recognition (ICDAR)*, 1015–1022.

A Prompts

This appendix and the schema of Appendix B are what this paper discloses in place of a code release (see the Ethical Statement). The Pass-1 prompt and the classification rule are reproduced verbatim, as the system emits them with the document content replaced by a placeholder; the other rules the body cites are summarized. **The complete text of all three prompts is provided as supplementary material (`prompts.txt`).**

A.1 Pass 1: depth-map inference

Run once per document, on a layout-stratified sample of text blocks (§4.1). Its output is supplied to every detection call as the authority for what each nesting level is.

```
You are analyzing the structural skeleton of an early
↪ learning standards document. You will NOT extract
↪ individual elements -- only the document's nesting
↪ hierarchy.

Read the sample below and identify how many distinct
↪ nesting depths the document uses, what each depth
↪ looks like (its prefix/label pattern), and one
↪ concrete example. The DEEPEST depth is always the leaf
↪  (individual learning goals/foundations/benchmarks).

This document's depths must then be mapped to a
↪ canonical 4-level hierarchy:
- depth 1 → "domain"
- depth 2 → "strand"
```

```
- depth 3 → "sub_strand"
- deepest depth → "indicator"

Rules:
- Fill the canonical levels FROM THE TOP DOWN. The first
↪  depth is always "domain", the deepest depth is always
↪  "indicator", and any depth in between takes the next
↪ canonical level in order: the depth directly under
↪ domain is "strand", the one below that is "sub_strand
↪ ". A document with fewer depths therefore drops the
↪ levels at the BOTTOM of that middle range -- never "
↪ strand".
- A 4-level document (domain > group > sub-group > leaf)
↪  maps to: domain, strand, sub_strand, indicator.
- A 3-level document (domain > group > leaf) maps to:
↪ domain, strand, indicator. There is NO sub_strand --
↪ the single middle level is a STRAND, never a
↪ sub_strand.
- A 2-level document (domain > leaf) maps to: domain,
↪ indicator. There is no strand and no sub_strand.
- Use depth POSITION, not the document's labels. If a
↪ document uses "Sub-Strand" as the label for the second
↪  level (directly under domain), it is still a STRAND
↪ in our canonical hierarchy -- and likewise a document
↪ that calls its single middle level "Section", "Concept
↪ ", "Goal", or anything else still maps that level to "
↪ strand".

Every line is prefixed with its page and the horizontal
↪ position it was laid out at: `[Page N | x=0.09]`,
↪ where x is the line's left edge as a fraction of the
↪ page width. Use x to work out whether a page uses a
↪ side-by-side layout: lines of one column share nearly
↪ the same x, and a jump to a clearly different x means
↪ a different column. Lines you judge to be in the same
↪ column read as one continuous cell, even when lines
↪ from other columns are interleaved between them. Treat
↪  parallel columns as repetitions of ONE depth, not as
↪ separate depths.

Be deterministic and conservative. Do not speculate. If
↪ you cannot tell whether two depths are distinct,
↪ assume they are the same depth.

Output ONLY a JSON object with this exact shape:
{
  "doc_depths": [
    {"depth": 1, "canonical_level": "domain",     "
↪ label_in_doc": "<what the doc calls this>", "
↪ prefix_pattern": "<regex-ish pattern e.g. 'ALL-CAPS
↪ HEADING' or 'N. <Title>: <desc>'>", "example": "<exact
↪  text from doc>"},
    {"depth": 2, "canonical_level": "strand|indicator",
↪ ...}
  ],
  "notes": "<one short sentence about the LEVEL LAYOUT
↪ only, e.g. 'age-band columns repeat one depth'. A
↪ later stage reads this as an instruction, so: never
↪ describe text below the deepest depth as examples/
↪ illustrations/anecdotes, and never say to ignore or
↪ skip anything. Empty string if nothing unusual.>"
}

DOCUMENT SAMPLE:
<<serialized document lines, one per row, each prefixed
  [Page N | x=<left edge as fraction of page width>]>>
```

A.2 Pass 2: per-chunk detection

The prompt opens with the canonical hierarchy, the layout-coordinate convention and the classification rule, then states nine numbered extraction rules and the output schema. Rule 1 requires every element to be backed by a heading line that literally appears in the chunk, so a chunk that opens part-way through a section emits the children alone and the parent is recovered in parsing.

```
CLASSIFICATION RULE:
- If a depth map is provided, look up each element's
↪ depth in the document and use the `canonical_level`
↪ from that depth. Do NOT reclassify based on prefix
↪ style.
- If no depth map is provided, classify by nesting
↪ POSITION, never by what the document calls a level.
```

Code resolution (rule 4) The longest rule. `code` is required and never `null`. A printed code always wins, and the rule orders three places to look for one: (i) inline on the heading line, including the `<Label> <id>` form for any label word; (ii) a caption beside the heading, such as a parenthetical or column header, copied verbatim; (iii) the shared leading prefix of the element's descendants' dotted codes, peeled one whole segment per level, agreed by at least two descendants, and never taken from a code that carries a structural label word. Only when all three fail is a code of at most five characters derived from the title: split on spaces and slashes, drop a fixed list of connector words, then take the first four letters of a single remaining word or the initials of up to five. That derivation is also executed in Python (§6, lesson 1).

A.3 Parsing

The parsing prompt states the output schema, then rules for composing each indicator's fully qualified code from its chain of ancestors. A code that is already a dotted path beginning with its own domain's code is used as is and never re-prefixed, at every level. A `<Label> <id>` code (`Benchmark 1.1`) contributes only its identifier, for any label word. Peeling ancestors off a descendant's code stops where the printed namespace stops, so a level outside it builds its code from its own heading's identifier (Nevada's strand `SS.1` above sub-strand `SS.ID`). Side-by-side columns are disambiguated by a suffix on the indicator code only: a month range for age columns (`.36-48`), otherwise the column label's first four letters (`.DISC`). The shape repairs of §4.2 that collapse a duplicated parent segment or decline an unverifiable repair are deterministic Python applied to the model's output, not prompt text.

B Canonical Schema

Figure 2 gives the canonical schema of §3. Field names are those of the implementation's typed model; per-element verification state (`human_verified`, `verified_at`,

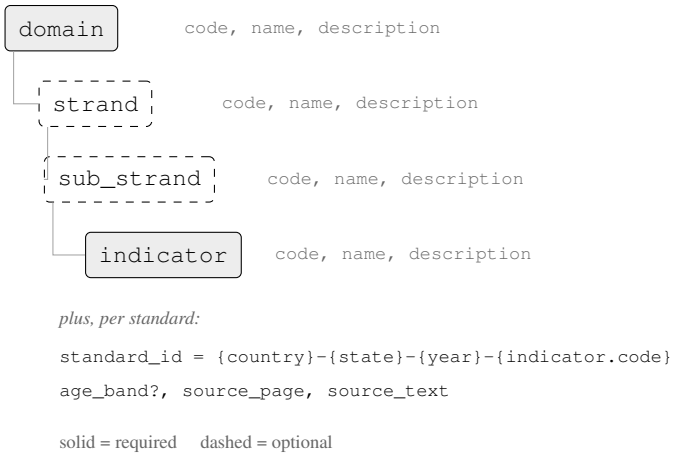


Figure 2: The canonical schema. Four hierarchy levels, of which `strand` and `sub_strand` are optional; each carries a code, a name and an optional description. A stored record is one indicator together with its fully resolved ancestry, not a node in a tree (§3).

`verified_by`) is stored at each level and omitted from the figure.

C Representative Golden Examples

The detector golden is a flat list of elements and the parser golden a nested record per standard. The two examples below show the conventions §4.2 turns on; descriptions are omitted, and codes and identifiers are reproduced in full.

KY detector golden: a `<Label> <id>` heading, whose label-and-id is the code

`level`	`sub_strand`
`code`	`Benchmark 1.1`
`title`	Maintains focus and sustains attention.

KY parser golden: the same element, its label form resolved to a domain-qualified code

`standard_id`	`US-KY-2021-AL.1.1.EASPT`
`domain.code`	`AL`
`domain.name`	Approaches to Learning
`strand.code`	`AL.1`
`strand.name`	Sustains attention and persists with challenging activities and experiences.
`sub_strand.code`	`AL.1.1`
`sub_strand.name`	Maintains focus and sustains attention.
`indicator.code`	`AL.1.1.EASPT`
`indicator.name`	Engages in an activity for a sustained period of time.

NV parser golden: a printed namespace that skips a level, so the sub-strand code does not extend the strand's

	n	**Range**	$\geq$ **0.95**
By hierarchy level			
domain	16	0.95–0.97	16/16
strand	51	0.92–0.97	48/51
sub-strand	49	0.92–0.97	47/49
indicator	263	0.90–0.97	246/263
By audit verdict (in-scope only)			
matched a golden	135	0.92–0.97	117/135
real, unannotated	117	0.92–0.97	115/117
real, split title	39	0.95–0.96	39/39
real, reprinted	5	0.95	5/5
invented	1	0.90	0/1
All	379	0.90–0.97	357/379

Table 5: Detector self-reported confidence over the 379 elements of the recorded run (subset tier). The verdict rows cover the 297 detections inside the domains a golden annotates; the other 82 were not audited. The prompt asks for ≥ 0.95 when the depth map clearly applies, 0.80–0.94 when the chunk is ambiguous, and < 0.70 when guessing; the score takes 5 distinct values in $[0.90, 0.97]$ and uses the bottom band never. **Nothing in the pipeline thresholds this score** (§6).

`standard_id`	`US-NV-2023-SS.ID.PK1`
`domain.code`	`SS`
`domain.name`	Social Studies
`strand.code`	`SS.1`
`strand.name`	Demonstrate a basic awareness of self as an individual, within the context of a group and community.
`sub_strand.code`	`SS.ID`
`sub_strand.name`	Individual Development and Cultural Identity
`indicator.code`	`SS.ID.PK1`
`indicator.name`	Identify and describe characteristics of self that are unique from others.

D Confidence Score Distribution

Table 5 gives the detector's confidence distribution by hierarchy level and by audit verdict, supporting §6. The score also drifts between runs: on an earlier freeze of the same corpus, 89.7% of indicators scored 0.95 or above, against 93.5% in the recorded run. The single confirmed hallucination is the lowest-scoring element (0.90), but with one invented element in the corpus no threshold's false-negative rate is measurable.

E Corpus

Table 6 identifies every source document, and Table 7 the pages each tier retains, so either tier can be reconstructed from the issuing agency's copy.

State	Role	Year	Publisher	Full	Trim.	Subset
CA	dev	2021	CA Dept. of Education	68	–	13
AZ	dev	2018	AZ Dept. of Education	217	65	15
TX	dev	2022	TX Education Agency	87	71	9
CO	dev	2020	CO Dept. of Early Childhood	41	41	10
NV	validation	2023	NV Dept. of Education	98	71	15
KY	validation	2021	Governor's Office of Early Childhood	120	52	8

Table 6: Corpus provenance. *Full* is the published PDF; *Trim.* removes non-standards matter while retaining every standard; *Subset* is the one-to-two-domain excerpt the headline quality measurements use, and is a genuine reduction in coverage. Colorado's *Full* is its 41-page Ages 3–5 publication. These are third-party state-agency documents, identified here rather than redistributed (see the Ethical Statement).

State	Tier	pp	Retained PDF pages
AZ	trimmed	65	2, 21, 23–28, 41, 43–50, 66, 70–82, 107, 110–120, 139, 142–145, 156, 158–162, 179, 181–185, 198, 201–204 (of 217)
AZ	subset	15	2, 21, 23, 66, 70–78, 81–82 (of 217)
CA	subset	13	1–5, 11–13, 18–22 (of 68)
CO	trimmed	41	1–41 (of 41)
CO	subset	10	1, 3–9, 11–12 (of 41)
TX	trimmed	71	1, 9–65, 69–81 (of 87)
TX	subset	9	1, 9–13, 59–61 (of 87)
NV	trimmed	71	1, 3, 16–84 (of 98)
NV	subset	15	1, 3, 23–26, 54–57, 61–65 (of 98)
KY	trimmed	52	1, 52–102 (of 120)
KY	subset	8	1, 52–55, 65, 71–72 (of 120)

Table 7: Pages each tier retains, as indices into the published PDF file. Printed page numbers differ by each document's front matter (Kentucky's PDF page 52 is printed page 39). Every tier page was matched by text to a published page. California has no trimmed tier.